Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université des Sciences et de la Technologie Houari Boumediene

Faculté d'informatique

Département IA & SD

MODULE : complexité

RAPPORT TP02 : RECHERCHE D'ÉLÉMENTS

MEMBRES DU GROUPE :

-ZIKIOU FATIMA AMIRA.

-BOUCHRA SKAKRI.

-SELMA BOUKACEM.

-RAYAN SENNIA.

Partie A : Recherche d'un élément

1. Tableau de n valeurs non triées

a. Fonction rechElets_TabNonTries

La fonction parcourt le tableau case par case jusqu'à trouver x ou atteindre la fin. Elle renvoie 1 si x existe, 0 sinon.

```
int rechElets_TabNonTries(int T[],int n,int x){
    int i=0;
    bool trouver;
    trouver=false;
    while (i<n && trouver==false){
        if (T[i]==x){
            trouver=true;

            printf("l'element %d est trouve dans le tableau a la posistion %d
\n",x,i+1);

        }
        else{i++;}

    }
    if(trouver==false){
     printf("L'element %d n'existe pas dans le tableau\n", x);

    }
    return trouver;

}
```

b. Complexité théorique

- **Meilleur cas :** x se trouve en première case. Une seule comparaison est faite, donc T(n) = 1 = **O(1)**.
- **Pire cas :** x est absent (ou en dernière case). Les n cases sont examinées, donc T(n) = n comparaisons, soit **O(n)**.

2. Tableau de n valeurs triées

2.1 Recherche séquentielle

a. Fonction rechElets_TabTries

Le tri permet de n'explorer qu'une moitié du tableau. On compare d'abord x à T[n/2] : si x est plus petit, on parcourt séquentiellement la première moitié (cases 0 à n/2 − 1), sinon la seconde moitié (cases n/2 à n − 1).

```
int rechElets_TabTries(int T[],int n,int x){
    int i,t;
    bool trouver;
    trouver=false;

    if(x<T[n/2]){
      i=0;
      t=n/2;
    }else{
```

```
        i=(n/2);
        t=n;
    }
    while (i<t && trouver==false){
        if (T[i]==x){
            trouver=true;

            printf("l'element %d est trouve dans le tableau a la posistion %d
\n",x,i+1);

        }
        else{i++;}

    }
    if(trouver==false){
     printf("L'element %d n'existe pas dans le tableau\n", x);

    }
    return trouver;

}
```

b. Complexité théorique

- **Meilleur cas :** x = T[0]. Après la comparaison avec T[n/2], x est trouvé dès la première case : 2 comparaisons, donc **O(1)**.
- **Pire cas :** x est plus grand que tous les éléments (ou absent de la seconde moitié). Il faut 1 comparaison, puis n/2 comparaisons : T(n) = n/2 + 1, soit **O(n)**. Le gain par rapport au tableau non trié est un facteur 2 seulement : seule la constante change.

2.2 Recherche dichotomique

a. Fonction rechElets_Dicho

À chaque itération, on compare x à l'élément du milieu T[milieu] et on élimine la moitié de l'intervalle qui ne peut pas contenir x. La fonction renvoie la position de x (à partir de 1) ou −1 s'il est absent. Elle est nommée rechDichotomique dans notre code.

```
int rechDichotomique(int T[], int n, int x)
{
    int gauche = 0;
    int droite = n - 1;
    int milieu;

    while (gauche <= droite)
    {
        milieu = (gauche + droite) / 2;

        if (T[milieu] == x) {
            return milieu + 1;
        }
        if (x < T[milieu])
            droite = milieu - 1;

        else
            gauche = milieu + 1;
    }

    return -1;
}
```

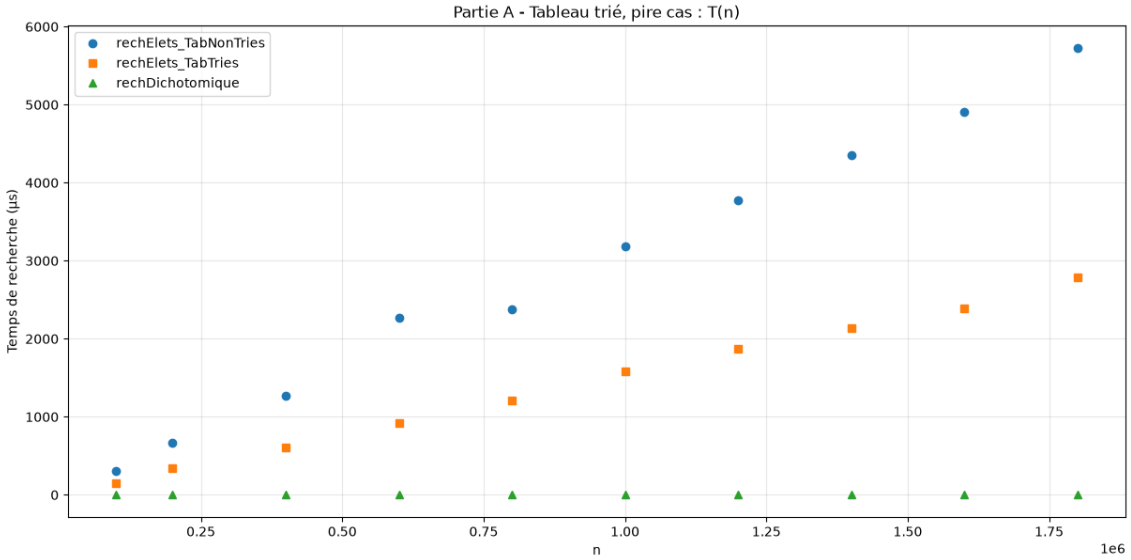
b. Complexité théorique

- **Meilleur cas :** x est l'élément du milieu. Il est trouvé à la première itération : T(n) = 1 = **O(1)**.
- **Pire cas :** x est absent (ou trouvé à la dernière itération). La taille de l'intervalle passe de n à n/2, n/4, …, 1, puis 0. Le nombre d'itérations est au plus $\lfloor \log_2 n \rfloor + 1$, donc **O(l**

3. Temps de recherche mesurés

n		100 000	200 000	400 000	600 000	800 000	1 000 000	1 200 000	1 400 000	1 600 000	1 800 000
EletsNonTriés	Meilleur cas	0,0058	0,0052	0,0078	0,0068	0,0056	0,0054	0,0052	0,0050	0,0062	0,0054
	Pire cas	301,50	672,00	1 274,00	2 273,27	2 376,00	3 190,00	3 771,08	4 359,15	4 904,00	5 729,73
EletsTriés	Meilleur cas	0,0076	0,0060	0,0092	0,0100	0,0064	0,0068	0,0070	0,0070	0,0070	0,0070
	Pire cas	149,00	343,00	606,00	918,92	1 204,00	1 585,00	1 867,47	2 133,80	2 392,00	2 792,79
EletsTriésDicho	Meilleur cas	0,0070	0,0075	0,0060	0,0070	0,0065	0,0065	0,0065	0,0065	0,0060	0,0065
	Pire cas	0,0970	0,1145	0,1015	0,1040	0,1090	0,1085	0,1145	0,1170	0,1150	0,1145

4. Variation de T(n) des trois fonctions (tableau trié, pire cas)



5. constatons-nous :

- rechElets_TabNonTries : le temps augmente linéairement avec n (de 302 µs pour n = 10^5 à 5 730 µs pour n = $1{,}8 \cdot 10^6$). C'est bien du O(n), car dans le pire cas on parcourt tout le tableau.
- rechElets_TabTries : le temps augmente aussi linéairement mais il est environ 2,1 fois plus petit, parce qu'on ne parcourt qu'une moitié du tableau. La complexité reste O(n).
- rechDichotomique : le temps reste presque constant (environ 0,11 µs) même quand n devient grand, ce qui correspond à O(log n). Les petites différences viennent des mesures.
- Pour n = $1{,}8 \cdot 10^6$, la dichotomie est environ 24 391 fois plus rapide que la recherche séquentielle triée et 50 041 fois plus rapide que la non triée.
- Au meilleur cas, les trois fonctions ont un temps très petit (environ 0,005 à 0,01 µs) qui ne dépend pas de n : c'est du O(1).

Quelques valeurs ne sont pas parfaitement alignées (par exemple n = 600 000), sans doute à cause de la précision de clock() et de l'état de la machine pendant le test.

Conclusion :

pour un grand tableau, la dichotomie est la meilleure méthode, mais il faut que le tableau soit trié.

Partie B : Recherche du maximum et du minimum

1. Approche naïve

a. Fonction MaxEtMinA

On initialise min et max avec T[0], puis on parcourt les éléments de 1 à n − 1. Chaque élément est comparé au minimum courant, puis au maximum courant.

```
void MaxEtMinA(int T[],int n, int *min, int *max){
    int i=0;
    compteur=0;
    *min=T[0];
    *max=T[0];
    for(i=1;i<n;i++){
        compteur++;
        if(T[i]<*min){
            *min=T[i];
        }

        compteur++;

        if(T[i]>*max){
            *max=T[i];
        }
    }
    printf("Min = %d\n", *min);
    printf("Max = %d\n", *max);
    printf("Comparaisons = %d\n", compteur);

}
```

b. Complexité théorique au pire cas

Pour chacun des n − 1 éléments restants, on fait 2 comparaisons (une avec min, une avec max), quelle que soit la disposition des valeurs. Le nombre de comparaisons est donc **2(n − 1) = 2n − 2**, soit **O(n)**.

2. Algorithme plus efficace (par paires)

a. Fonction MaxEtMinB

Les éléments sont comparés deux à deux. Dans chaque paire (T[i], T[i+1]), on place le plus grand en case paire et le plus petit en case impaire (échange si nécessaire). Ensuite, le minimum est cherché parmi les petits éléments et le maximum parmi les grands. Si n est impair, le dernier élément, non comparé dans la première phase, est comparé à min et à max.

```
void MaxEtMinB(int T[], int n) {
    int i, min, max, c;

    compteur = 0;

    // Initialisation avec la première paire
    compteur++;

    if(T[0] < T[1]) {
        min = T[0];
```

```
        max = T[1];
    }
    else {
        min = T[1];
        max = T[0];
    }

    // Traitement des autres paires
    for(i = 2; i+1 < n; i += 2) {
        compteur++;

        if(T[i] < T[i+1]) {
            c = T[i];
            T[i] = T[i+1];
            T[i+1] = c;
        }

        compteur++;
        if(T[i+1] < min) {
            min = T[i+1];
        }

        compteur++;
        if(T[i] > max) {
            max = T[i];
        }
    }

    // Traitement du dernier élément si n est impair
    if(n % 2 != 0) {
        compteur++;

        if(T[n-1] < min) {
            min = T[n-1];
        }

        compteur++;

        if(T[n-1] > max) {
            max = T[n-1];
        }
    }

    printf("Min = %d\n", min);
    printf("Max = %d\n", max);
    printf("Comparaisons = %d\n", compteur);
}
```

b. Complexité théorique au pire cas

- Comparaisons des paires : $\lfloor n/2 \rfloor$.
- Minimum parmi les $\lfloor n/2 \rfloor$ petits éléments : $\lfloor n/2 \rfloor - 1$ comparaisons.
- Maximum parmi les $\lfloor n/2 \rfloor$ grands éléments : $\lfloor n/2 \rfloor - 1$ comparaisons.
- Si n est impair, le dernier élément coûte 2 comparaisons de plus.

Pour n pair : $n/2 + 2(n/2 - 1) = 3n/2 - 2$. Pour n impair : $(n - 1)/2 + 2((n - 1)/2 - 1) + 2 = 3(n - 1)/2$. Dans les deux cas, le nombre de comparaisons est $\lceil 3n/2 \rceil - 2$, soit **O(n)**. Il est inférieur à $2(n - 1)$.

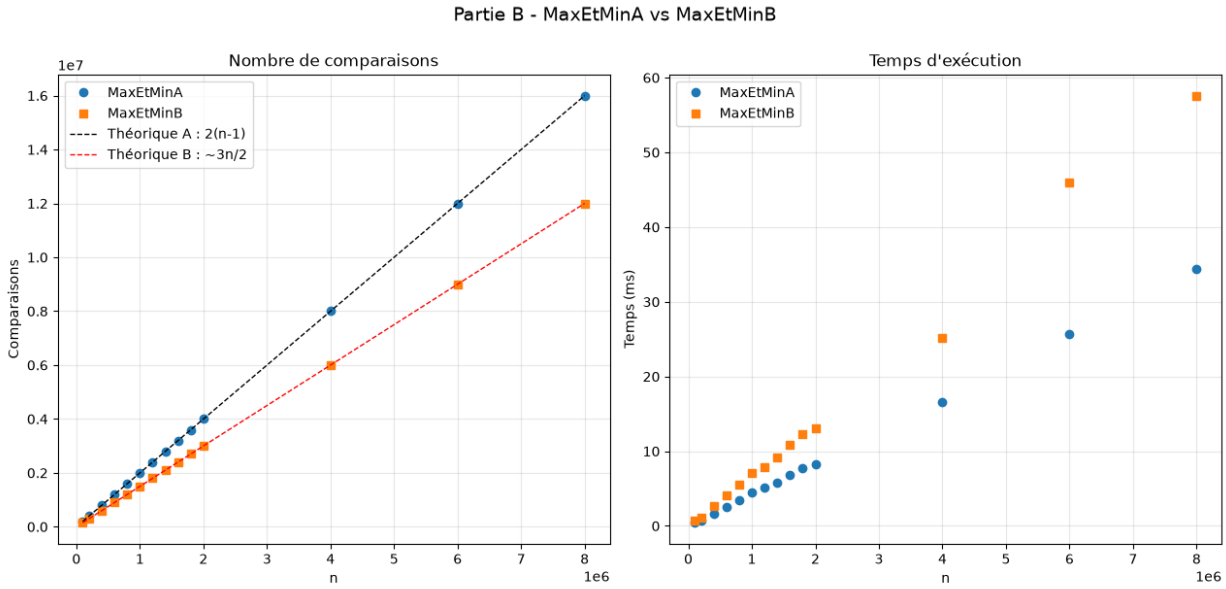
3. Comparaison du nombre de comparaisons

n	cmp A (mesuré)	2(n−1)	cmp B (mesuré)	⌈ 3n/2⌉ −2	cmp B / cmp A
100 000	199 998	199 998	149 998	149 998	0,750
200 000	399 998	399 998	299 998	299 998	0,750
400 000	799 998	799 998	599 998	599 998	0,750
600 000	1 199 998	1 199 998	899 998	899 998	0,750
800 000	1 599 998	1 599 998	1 199 998	1 199 998	0,750
1 000 000	1 999 998	1 999 998	1 499 998	1 499 998	0,750
1 200 000	2 399 998	2 399 998	1 799 998	1 799 998	0,750
1 400 000	2 799 998	2 799 998	2 099 998	2 099 998	0,750
1 600 000	3 199 998	3 199 998	2 399 998	2 399 998	0,750
1 800 000	3 599 998	3 599 998	2 699 998	2 699 998	0,750
2 000 000	3 999 998	3 999 998	2 999 998	2 999 998	0,750
4 000 000	7 999 998	7 999 998	5 999 998	5 999 998	0,750
6 000 000	11 999 998	11 999 998	8 999 998	8 999 998	0,750
8 000 000	15 999 998	15 999 998	11 999 998	11 999 998	0,750

Les valeurs mesurées sont les mêmes que les valeurs théoriques pour tous les n. MaxEtMinB fait 25 % de comparaisons en moins (rapport 0,75), car chaque paire coûte 3 comparaisons au lieu de 4.

4. Complexité temporelle expérimentale

n	Temps MaxEtMinA (ms)	Temps MaxEtMinB (ms)	Rapport B / A
100 000	0,429	0,681	1,59
200 000	0,724	1,053	1,45
400 000	1,634	2,632	1,61
600 000	2,532	4,033	1,59
800 000	3,496	5,580	1,60
1 000 000	4,455	7,040	1,58
1 200 000	5,157	7,831	1,52
1 400 000	5,782	9,218	1,59
1 600 000	6,800	10,888	1,60
1 800 000	7,721	12,288	1,59
2 000 000	8,200	13,050	1,59
4 000 000	16,520	25,140	1,52
6 000 000	25,697	46,030	1,79
8 000 000	34,400	57,600	1,67



Interprétation

- Comparaisons : les points mesurés sont sur les droites théoriques $2(n-1)$ et $3n/2$. MaxEtMinB est meilleur de ce côté.
- Temps : les deux courbes sont linéaires, donc les deux algorithmes sont en O(n). Mais MaxEtMinB est plus lent que MaxEtMinA, entre 1,45 et 1,79 fois (environ 1,6 en moyenne). Pour $n = 8 \cdot 10^6$: 34,4 ms pour A et 57,6 ms pour B.

Explication possible : MaxEtMinB fait aussi des échanges dans le tableau et le test T[i] < T[i+1] sur des valeurs aléatoires est difficile à prévoir pour le processeur, alors que dans MaxEtMinA les tests avec min et max sont presque toujours faux. On n'a pas vérifié cette explication par une mesure.

Conclusion : MaxEtMinB diminue bien le nombre de comparaisons (3n/2 au lieu de 2n), mais dans nos mesures il n'est pas plus rapide. Le nombre de comparaisons ne suffit donc pas pour prévoir le temps réel.

Conclusion générale

Ce TP nous a permis de vérifier les complexités théoriques par des mesures. En recherche d'un élément, la dichotomie en O(log n) est beaucoup plus rapide que les recherches séquentielles en O(n), à condition que le tableau soit trié. Pour le maximum et le minimum, la méthode par paires fait moins de comparaisons mais n'est pas plus rapide en temps dans nos tests.